

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <ctype.h>
#include <stdbool.h>

#define MAX_SIZE 20      // Maximum supported grid size (user input must be <= this)

/* 8 possible directions: right, left, down, up, and 4 diagonals */
const int DIRECTIONS[8][2] = {
    {0, 1}, /* horizontal right */
    {0, -1}, /* horizontal left */
    {1, 0}, /* vertical down */
    {-1, 0}, /* vertical up */
    {1, 1}, /* diagonal down-right */
    {1, -1}, /* diagonal down-left */
    {-1, 1}, /* diagonal up-right */
    {-1, -1} /* diagonal up-left */
};

/* Initialize grid with placeholder dots (empty cells) */
void init_grid(char grid[][MAX_SIZE], int size) {
    for (int r = 0; r < size; r++) {
        for (int c = 0; c < size; c++) {
            grid[r][c] = '.'; // '.' means empty
        }
    }
}

/* Try to place a single word in a random position and direction.
 * Returns true if successfully placed, false after too many failed attempts. */
bool place_word(char grid[][MAX_SIZE], int size, const char *word) {
    int len = (int)strlen(word);
    if (len == 0 || len > size) return false;

    /* Try up to 1000 random placements */
    for (int attempt = 0; attempt < 1000; attempt++) {
        int r = rand() % size; // random starting row
        int c = rand() % size; // random starting column
        int d = rand() % 8; // random direction index

        int dr = DIRECTIONS[d][0];
        int dc = DIRECTIONS[d][1];

        /* Check if the word fits in this position and direction */
        bool can_place = true;
        for (int i = 0; i < len; i++) {

```

```

int nr = r + i * dr;
int nc = c + i * dc;

/* Boundary check */
if (nr < 0 || nr >= size || nc < 0 || nc >= size) {
    can_place = false;
    break;
}

/* Overlap check: must be empty or already match the required letter */
if (grid[nr][nc] != '.' && grid[nr][nc] != word[i]) {
    can_place = false;
    break;
}
}

if (can_place) {
    /* Place the word */
    for (int i = 0; i < len; i++) {
        int nr = r + i * dr;
        int nc = c + i * dc;
        grid[nr][nc] = word[i];
    }
    return true;
}
}
return false; // Could not place after many tries
}

/* Fill all remaining '.' cells with random uppercase letters A-Z */
void fill_grid(char grid[][MAX_SIZE], int size) {
    for (int r = 0; r < size; r++) {
        for (int c = 0; c < size; c++) {
            if (grid[r][c] == '.') {
                grid[r][c] = 'A' + (rand() % 26);
            }
        }
    }
}

/* Print the grid. If highlight[][] is set for a cell, print that letter in green. */
void print_grid(const char grid[][MAX_SIZE], int size, const bool highlight[][MAX_SIZE]) {
    for (int r = 0; r < size; r++) {
        for (int c = 0; c < size; c++) {
            if (highlight[r][c]) {
                /* ANSI escape: green foreground */
                printf("\x1b[32m%c\x1b[0m ", grid[r][c]);
            } else {

```

```

        printf("%c ", grid[r][c]);
    }
}
printf("\n");
}
}

```

/* Search for a word in all 8 directions from every cell.

* If found, sets highlight[][] for the matching letters and returns true. */
bool find_word(const char grid[][MAX_SIZE], int size, const char *word, bool highlight[][MAX_SIZE]) {

```

    int len = (int)strlen(word);
    if (len == 0) return false;

```

/* Try every possible starting position */

```

for (int r = 0; r < size; r++) {
    for (int c = 0; c < size; c++) {
        /* Try every direction */
        for (int d = 0; d < 8; d++) {
            int dr = DIRECTIONS[d][0];
            int dc = DIRECTIONS[d][1];

```

```

            bool matches = true;

```

/* Check each letter along this path */

```

for (int i = 0; i < len; i++) {
    int nr = r + i * dr;
    int nc = c + i * dc;

```

```

    if (nr < 0 || nr >= size || nc < 0 || nc >= size || grid[nr][nc] != word[i]) {
        matches = false;
        break;
    }
}

```

```

if (matches) {

```

/* Found! Mark the letters for highlighting */

```

for (int i = 0; i < len; i++) {
    int nr = r + i * dr;
    int nc = c + i * dc;
    highlight[nr][nc] = true;
}

```

```

return true; // Only highlight the first occurrence found
}

```

```

}
}
}
return false; // Word not found anywhere
}

```

```

/* Main program */
int main(void) {
    srand((unsigned int)time(NULL)); // Seed random number generator

    int size;
    printf("=== Word Search Generator & Solver ===\n");
    printf("Enter grid size (5-20): ");
    if (scanf("%d", &size) != 1 || size < 5 || size > MAX_SIZE) {
        printf("Invalid size! Using default 12x12.\n");
        size = 12;
    }

    char grid[MAX_SIZE][MAX_SIZE];
    bool highlight[MAX_SIZE][MAX_SIZE];

    /* Step 1: Initialize empty grid */
    init_grid(grid, size);

    /* Step 2: Place fixed list of words */
    const char *word_list[] = {"QUIZ", "EXAM", "MID", "FINAL"};
    int num_words = sizeof(word_list) / sizeof(word_list[0]);

    printf("Placing words...\n");
    for (int i = 0; i < num_words; i++) {
        if (!place_word(grid, size, word_list[i])) {
            printf(" Warning: Could not place \"%s\"\n", word_list[i]);
        } else {
            printf(" Placed \"%s\"\n", word_list[i]);
        }
    }

    /* Step 3: Fill remaining cells with random letters */
    fill_grid(grid, size);

    /* Clear highlight array */
    memset(highlight, 0, sizeof(highlight));

    /* Step 4: Print the generated puzzle */
    printf("\nGenerated Word Search Grid (%dx%d):\n", size, size);
    print_grid(grid, size, highlight);

    /* Step 5: Ask user for a word to search */
    char search[20];
    printf("\nEnter a word to search for (uppercase recommended): ");
    scanf("%19s", search);

    /* Convert input to uppercase for consistency */

```

```
for (int i = 0; search[i]; i++) {
    search[i] = (char)toupper(search[i]);
}

/* Clear highlight for the new search */
memset(highlight, 0, sizeof(highlight));

/* Step 6: Search and show result */
if (find_word(grid, size, search, highlight)) {
    printf("\n✅ Word \"%s\" FOUND!\n", search);
    printf("Highlighted grid:\n");
    print_grid(grid, size, highlight);
} else {
    printf("\n❌ Word \"%s\" not found.\n", search);
}

printf("\nThank you for using the Word Search Generator & Solver!\n");
return 0;
}
```